

Monit Operations Standard

SELinux-enforcing host monitoring, local-only control, safe log rotation, and evidence-driven operations

Version 1.0 | August 31, 2026

Canonical source digest: 744ba260a9a4e43037a4

This component installs and configures a client-neutral Monit baseline for RHEL-family virtual machines. It keeps SELinux enforcing, uses the signed EPEL package, rotates a private file log without restarting Monit, exposes the control API only on IPv4 loopback, and leaves application restart ownership with systemd unless a deployment explicitly approves a different action.

Hostnames, notification recipients, mail or M/Monit credentials, application names, addresses, and deployment evidence do not belong in this public repository. Supply them as protected deployment fragments and retain their source and acceptance evidence in the relevant private system of record.

Supported platform and version policy

The installer supports the repository's RHEL-family baseline: RHEL, Rocky Linux, and CentOS Stream major versions 9 and 10 on x86-64 or ARM64. It enables the appropriate EPEL repository through the shared platform helper and installs the repository's current signed `monit` package.

The control syntax has a Monit 5.33 floor. The baseline was last reviewed with Monit 6.0.0 from EPEL on August 31, 2026. The installer deliberately follows the supported EPEL package rather than pinning an old build; the verifier rejects a downgrade below the syntax floor and checks the installed binary and vendor systemd unit with RPM verification.

Security and ownership model

- `/etc/monitrc` is `root:root` mode `0600`. Setup renders a random 256-bit hexadecimal password for the local control client and preserves that value on later managed reruns.
- TCP 2812 binds only to `127.0.0.1`. There is no public Monit dashboard or management port. Root uses the protected control file for `monit summary`, `monit status`, and other client commands.
- `/etc/monit.d/*.conf` fragments are `root:root` mode `0600`. Existing matching fragments are preserved. A requested fragment is installed by its basename, and duplicate or unsafe names fail before host changes.
- The public resource fragment alerts on sustained load, CPU, memory, swap, root-filesystem space, and inode pressure. It contains no start, stop, or restart action.
- systemd owns Monit's own recovery through `Restart=on-failure`. The drop-in creates private runtime, state, and log directories and applies `UMask=0077`.
- Notification transport and recipients are deployment secrets. Place them in a separate protected fragment; never add them to the public template, a command line, a rendered plan, or an evidence record.

Monit is intentionally a privileged observer. A general monitor may need to read process state, sockets, files, filesystems, and service-specific paths, so this baseline does not invent a broad local SELinux policy module or run `audit2allow`. It uses the distribution's default file contexts, restores those labels during setup, requires Enforcing mode, rejects `unconfined_t`, and fails acceptance if the audit log contains a Monit AVC denial since boot. Investigate and narrowly resolve a real denial rather than disabling SELinux or installing generated allow rules without review.

Installed layout

Path	Purpose and identity
/etc/monitrc	Managed global control file; root:root mode 0600
/etc/monit.d/10-system.conf	Client-neutral alert-only VM checks; root:root mode 0600
/etc/monit.d/*.conf	Preserved or explicitly supplied deployment fragments; root:root mode 0600
/etc/logrotate.d/monit	Daily/25 MiB rotation, 14 archives, compression, and SIGHUP reopen
/etc/systemd/system/monit.service.d/10-baseline.conf	Runtime directories, umask, restart, and signal reload policy
/var/log/monit/monit.log	Private Monit file log; root:root mode 0600
/var/lib/monit	Persistent ID, state, and queued events; root:root mode 0700
/run/monit	Runtime PID directory; root:root mode 0750
/usr/local/bin/verify-monit	Installed read-only acceptance verifier

The main control file includes only `/etc/monit.d/*.conf`. Staging files must therefore use a different suffix or a directory outside `/etc/monit.d`; a partially written `.conf` must never become visible to the live include glob.

Install

Review package and repository actions without changing the host:

```
monit/install --plan
```

Install Monit and its audit, SELinux, logrotate, OpenSSL, process, and network inspection dependencies:

```
sudo monit/install
```

The installer preserves an existing service's active/enabled state. A fresh installation remains stopped and disabled until setup validates a complete candidate. It also installs `verify-monit` and the exact public policy sources the installed verifier compares against.

Render and review setup

Render a self-contained install tree without touching the host:

```
monit/setup --output /tmp/monit-baseline
```

Add one or more deployment fragments to the candidate by their safe basename:

```
monit/setup --output /tmp/monit-host \  
--fragment /private/reviewed/50-notifications.conf \  
--fragment /private/reviewed/70-service.conf
```

The output contains a generated local control credential and is therefore a protected deployment artifact. Review it, keep it out of source control, and remove ordinary staging copies after promotion.

Preview the live action separately:

```
monit/setup --plan \  
--fragment /private/reviewed/50-notifications.conf \  
--fragment /private/reviewed/70-service.conf
```

Apply and adopt

For a fresh package configuration or a host already managed by this standard:

```
sudo monit/setup \  
--fragment /private/reviewed/50-notifications.conf \  
--fragment /private/reviewed/70-service.conf
```

Setup assembles the candidate with every existing live *.conf fragment, parses the full tree, validates logrotate, takes recoverable copies of every managed target, installs atomically by file, restores SELinux labels, reloads systemd, and then either reloads an active Monit daemon or enables and starts a fresh one. It runs the complete verifier before committing the transaction. Failure restores the managed files and prior service enablement/activity state.

The EPEL package's unmodified example /etc/monitrc may be replaced safely. Any other unmanaged modification is a migration boundary because it may hold notification secrets or service policy. Move those directives to reviewed root-only .conf fragments, render the combined candidate, and only then make the adoption explicit:

```
sudo monit/setup --replace-main \  
--fragment /private/reviewed/50-notifications.conf
```

--replace-main authorizes replacement of only /etc/monitrc; it does not delete unrelated fragments. Keep the setup backup until notification delivery and all service checks have passed acceptance.

Deployment fragment contract

A deployment fragment must be non-symbolic, end in `.conf`, and use a basename containing only letters, numbers, dots, underscores, and hyphens. The reserved name `10-system.conf` cannot be supplied. Keep global notification policy in an earlier fragment and service checks in later clearly named fragments, but do not rely on wildcard include order for cross-file definitions: Monit documents glob inclusion as unsorted. A fragment must be independently meaningful or use an explicit full-path include when order is required.

Prefer alert-only checks. A service already managed by `systemd` should not also be restarted by Monit without an explicit failure-amplification analysis. In particular, a shared database or network outage must not trigger restart loops across otherwise healthy applications.

Before promotion, parse the exact assembled tree with the target host's Monit binary. Avoid control forms introduced after the supported floor. A route or socket check should use explicit timeouts, consecutive failed cycles, and a recovery notification. Do not stop a production service merely to test alert delivery; use a temporary controlled failing check and then observe recovery.

Log rotation and SELinux

Monit writes `/var/log/monit/monit.log`. Logrotate runs daily or when the log exceeds 25 MiB, retains 14 dated archives, compresses after one cycle, creates the new log as `root:root` mode `0600`, and sends `SIGHUP` to the active service. The upstream Monit signal contract reopens logs and rereads configuration, so rotation does not restart the daemon or a monitored application.

Setup restores the policy-default labels on configuration, `systemd`, state, runtime, and log paths. Run the verifier after first start and after adding a check that reads a new service-specific path. If it reports a denial, collect the exact AVC, confirm the intended read or action, and prefer an existing distribution label or interface. Do not use permissive mode as a deployment step.

Verify and collect acceptance evidence

Run the installed verifier as root:

```
sudo /usr/local/bin/verify-monit
```

Acceptance additionally requires deployment-specific checks:

1. `monit summary` and every intended `monit status NAME` report monitored and healthy after at least three collection cycles.
2. The public or private service path being monitored has the intended failure meaning and does not expose credentials in output.
3. A controlled failure alert and its recovery notification are received through the protected transport.
4. Monit's main PID and every monitored application's main PID are unchanged by configuration promotion and log rotation.
5. `ausearch -m AVC,USER_AVC -ts boot -c monit` returns no denial.
6. The only TCP 2812 listener is `127.0.0.1:2812`; no host or cloud firewall rule exposes it.
7. The private evidence record captures package NEVRA, policy revision, sanitized status, alert receipt, PID continuity, and rollback owner.

Rollback

The setup transaction restores managed files automatically when validation or activation fails. For a later operator rollback, restore the retained copies of `/etc/monitrc`, the changed `.conf` fragments, the logrotate policy, and the `systemd` drop-in; then run:

```
sudo systemctl daemon-reload
sudo monit -t -c /etc/monitrc
sudo systemctl reload monit.service
```

If Monit was a fresh deployment and must be removed from service, disable and stop `monit.service` first. Package removal is a separate reviewed action; do not delete state, logs, fragments, or evidence until their retention decision is recorded.

Reproduce the PDF handoff

The Markdown file is canonical. Validate it without ReportLab:

```
python3 monit/build-operations-standard-pdf.py --check
```

Build the delivery artifact when ReportLab is available:

```
python3 monit/build-operations-standard-pdf.py
```

Render every page with Poppler and inspect the PNGs before delivery. Text extraction alone does not prove that tables, code, or page transitions are legible.

Official references

- [Monit manual](#)
- [Fedora package metadata for Monit](#)
- [Red Hat SELinux administration](#)
- [logrotate manual](#)